

CAPÍTULO 1: FUNDAMENTOS

Programación con Python | Material de consulta



1. Antes de escribir código

NO empieces programando inmediatamente. Primero preguntate:

- ¿Qué datos necesito? → ¿Qué información me da el usuario?
- ¿Qué tengo que hacer? → ¿Qué calculo, modifico o analizo?
- ¿Qué tengo que mostrar? → ¿Cuál debe ser la salida?

```
nombre = input("Nombre: ")
edad = int(input("Edad: "))

salario_total = horas * valor_por_hora

print("Salario:", salario_total)
```

2. ¿Necesito pedir información?

Si el ejercicio dice solicitar, pedir, ingresar o introducir, probablemente necesitás input().

```
nombre = input("Ingrese su nombre: ")
edad = int(input("Ingrese su edad: "))
altura = float(input("Ingrese su altura: "))
```

⚠ Importante: input() devuelve texto (str). Si necesitás hacer operaciones matemáticas, normalmente tenés que convertirlo.

3. ¿Necesito guardar información?

Usá una variable. Elegí nombres que indiquen claramente qué información almacenan.

```
nombre = "Franco"
edad = 16
precio = 2500.50
salario_total = 25000
```

Mejor: precio_total Evitar: x o a cuando podés usar un nombre descriptivo.

4. ¿Necesito mostrar algo?

Usá print().

```
print(nombre)
print("Hola", nombre)
print("Edad:", edad)

total = precio * cantidad
print("Total:", total)
```

5. ¿Necesito hacer una operación?

Necesito...	Operador
Sumar	+
Restar	-
Multiplicar	*
Dividir	/
División entera	//
Obtener resto	%
Potencia	**

6. ¿Necesito comparar?

Quiero saber si...	Operador
son iguales	<code>==</code>
son distintos	<code>!=</code>
uno es mayor	<code>></code>
uno es menor	<code><</code>
mayor o igual	<code>>=</code>
menor o igual	<code><=</code>

7. ¿Estoy trabajando con texto?

```
nombre = "Martina"

len(nombre)          # cantidad de caracteres
nombre.upper()        # MAYÚSCULAS
nombre.lower()        # minúsculas
nombre.capitalize()   # Primera letra en mayúscula
nombre.strip()        # elimina espacios extremos
nombre.replace("a", "o") # reemplaza texto
nombre.find("a")       # busca y devuelve posición
nombre.startswith("Mar") # comienza con...
nombre.endswith(".com") # termina con...
```

8. ¿Necesito obtener un carácter?

Las posiciones empiezan en 0.

```
P Y T H O N
0 1 2 3 4 5

palabra[0] → P
palabra[3] → H
palabra[-1] → N
```

9. ¿Necesito obtener una parte del texto?

```
cadena[inicio:fin]

palabra = "Python"
palabra[0:3] → "Pyt"
palabra[:3] → desde el principio
palabra[3:] → hasta el final
palabra[::-1] → invertida
```

⚠ En `[inicio:fin]`, la posición `fin` no se incluye.

CAPÍTULO 2: ESTRUCTURA DE DATOS

Programación con Python | Material de consulta



1. ¿QUÉ ESTRUCTURA NECESITO?

Si el problema dice...	Probablemente necesito...
si ocurre...	if
si ocurre..., sino...	if / else
A, B o C...	if / elif / else
y además...	and
O...	or
no...	not
repetir hasta que...	while
repetir N veces...	for
recorrer una lista...	for
agregar / eliminar / ordenar...	métodos de listas
buscar / contar / sumar / mayor / menor...	algoritmos sobre listas
filas y columnas...	matriz
último en entrar, primero en salir	pila / LIFO
primero en entrar, primero en salir	cola / FIFO

2. CONDICIONALES

if

```
if condicion:
    instrucciones
```

if / else

```
if condicion:
    instrucciones
else:
    otras_instrucciones
```

if / elif / else

```
if condicion1:
    ...
elif condicion2:
    ...
else:
    ...
```

⚠ Python analiza de arriba hacia abajo. Cuando una condición es verdadera, ejecuta ese bloque y no continúa con los siguientes.

3. OPERADORES LÓGICOS

Operador	Significado	Ejemplo
and	Todas verdaderas	edad >= 18 and tiene_dni
or	Al menos una verdadera	es_profesor or es_directivo
not	Invierte el valor	not es_admin

Rango:

```
if 18 <= edad <= 65:
    print("Edad permitida")
```

4. WHILE

Usalo cuando no sabés exactamente cuántas veces se repetirá algo.

```
contador = 1

while contador <= 5:
    print(contador)
    contador += 1
```

☐ Un while debe poder terminar. Preguntate: ¿qué hace que la condición se vuelva falsa?

5. FOR Y RANGE

Usalo cuando conocés la cantidad de repeticiones o querés recorrer una colección.

```
for i in range(5):
    print(i)
```

`range(5)` produce 0, 1, 2, 3, 4. El 5 no está incluido.

```
range(5) → 0 1 2 3 4
range(3, 8) → 3 4 5 6 7
range(2, 11, 2) → 2 4 6 8 10
```

6. CONTADOR Y ACUMULADOR

Contador:

```
contador = 0
for elemento in lista:
    if condicion:
        contador += 1
```

Acumulador:

```
suma = 0
for elemento in lista:
    suma += elemento
```

7. LISTAS

Una lista guarda varios valores. Los índices comienzan en 0.

```
notas = [8, 7, 10, 6, 9]
notas[0] # primero
notas[2] # tercero
notas[-1] # último
len(notas) # cantidad
```

Las listas son mutables: sus elementos pueden modificarse.

```
notas[0] = 10
```

8. RECORRER UNA LISTA

Si solo necesitás valores:

```
for elemento in lista:
    print(elemento)
```

Si necesitás la posición:

```
for i in range(len(lista)):
    print(i, lista[i])
```

$i = \text{posición} \cdot \text{lista}[i] = \text{valor}$

9. MÉTODOS DE LISTAS

Método	¿Qué hace?
append(x)	Agrega al final
insert(i, x)	Inserta en una posición
remove(x)	Elimina por valor
pop(i)	Elimina por posición
pop()	Elimina el último
index(x)	Busca la posición
count(x)	Cuenta apariciones
sort()	Ordena
sort(reverse=True)	Ordena de mayor a menor
reverse()	Invierte
clear()	Vacía
copy()	Copia

10. IN

```
if "Pedro" in nombres:  
    print("Encontrado")
```

11. ALGORITMOS SOBRE LISTAS

Suma

```
suma = 0  
for numero in numeros:  
    suma += numero
```

Promedio

```
promedio = suma / len(lista)
```

Mayor

```
mayor = numeros[0]  
for numero in  
numeros:  
    if numero > mayor:  
        mayor = numero
```

Menor

```
menor = numeros[0]  
for numero in  
numeros:  
    if numero < menor:  
        menor = numero
```

12. BUSQUEDA LINEAL

Revisa los elementos uno por uno hasta encontrar el buscado.

```
encontrado = False
for elemento in lista:
    if elemento == buscado:
        encontrado = True

if encontrado:
    print("Existe")
else:
    print("No existe")
```

Buscar una posición:

```
indice = -1
for i in range(len(lista)):
    if lista[i] == buscado:
        indice = i
        break
```

`-1` puede representar "no encontrado". `break` termina el bucle.

13. MATRICES

Una matriz es una lista de listas.

```
notas = [
    [8, 9],
    [6, 7],
    [10, 8]
]

notas[0][0] # fila 0, columna 0
```

Para recorrerla necesitamos dos bucles:

```
for fila in notas:
    for elemento in fila:
        print(elemento)
```

Si necesitas índices:

```
for i in range(len(notas)):
    for j in range(len(notas[i])):
        print(notas[i][j])
```

$i = \text{fila}$ · $j = \text{columna}$

14. PILAS — LIFO

Last In, First Out → el último en entrar es el primero en salir.

```
pila = []
pila.append(10)
pila.append(20)
pila.append(30)
pila.pop() # sale 30
```

Ejemplos: Ctrl + Z, historial de navegación, llamadas de funciones.

15. COLAS — FIFO

First In, First Out → el primero en entrar es el primero en salir.

```
cola = []
cola.append("Ana")
cola.append("Luis")
cola.append("Pedro")
cola.pop(0) # sale Ana
```

16. PILA VS COLA

	PILA	COLA
Principio	LIFO	FIFO
Agregar	append()	append()
Eliminar	pop()	pop(0)
Sale primero	Último	Primero
Ejemplo	Pila de platos / Ctrl + Z	Fila de personas

17. PATRONES QUE TENÉS QUE RECONOCER

No memorices programas completos. Aprendé a reconocer estos patrones.

Contador

```
contador = 0
for elemento in lista:
    if condicion:
        contador += 1
```

Acumulador

```
suma = 0
for elemento in lista:
    suma += elemento
```

Mayor

```
mayor = lista[0]
for elemento in lista:
    if elemento > mayor:
        mayor = elemento
```

Menor

```
menor = lista[0]
for elemento in lista:
    if elemento < menor:
        menor = elemento
```

Buscar

```
encontrado = False
for elemento in lista:
    if elemento == buscado:
        encontrado = True
```

Buscar posición

```
indice = -1
for i in range(len(lista)):
    if lista[i] == buscado:
        indice = i
        break
```

Recorrer matriz

```
for fila in matriz:  
    for elemento in fila:  
        print(elemento)
```

Promedio

```
suma = 0  
for elemento in lista:  
    suma += elemento  
  
promedio = suma / len(lista)
```

18. ¿CÓMO RESUELVO UN EJERCICIO?

Paso	Pregunta
1	¿Qué datos tengo?
2	¿Dónde los guardo?
3	¿Tengo que tomar una decisión? → if
4	¿Tengo que repetir algo? → while / for
5	¿Trabajo con muchos datos? → lista
6	¿Necesito recorrerlos? → for
7	¿Necesito encontrar algo? → búsqueda
8	¿Necesito sumar? → acumulador
9	¿Necesito contar? → contador
10	¿Tengo filas y columnas? → matriz